Assignment Number:16.3

2303A51817

K.Akshay

Q.No.	Question

1	**Lab 16 – Database Design and Queries: Schema Design and SQL Generation**
	Lab Objectives:
	• Understand schema design principles for relational databases.
	• Use AI-assisted tools to generate **SQL schema definitions**.
	• Write and refine **SQL queries** (CRUD + aggregate operations).
	• Explore how AI can optimize queries and ensure normalization.
	
	Task 1 – Hospital Management Database
	Task:
	Create schema and queries for a **Hospital Management System**.
	Instructions:
	• Tables: Doctors, Patients, Appointments.
	• Use AI to define constraints (unique IDs, valid dates).
	• Generate queries:
	  • List all appointments for a specific doctor.
	  • Retrieve patient history by patient ID.
	  • Count total patients treated by each doctor.
	Expected Output:
	• Normalized schema and SQL queries with joins.
	Prompt :
	Create a normalized Hospital Management System schema with Doctors, Patients, and Appointments tables, enforce unique IDs and valid dates, then write SQL queries to list appointments for a doctor, retrieve patient history by ID, and count distinct patients per doctor using joins.
	Code :

```python
import sqlite3
import pandas as pd
from datetime import datetime, timedelta

# Connect to an in-memory SQLite database
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

print("Connected to in-memory SQLite database.")
```

```
Connected to in-memory SQLite database.
```

Create Tables

```python
create_tables_sql = """
CREATE TABLE Doctors (
    doctor_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    specialty VARCHAR(100),
    phone_number VARCHAR(15),
    email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Patients (
    patient_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE,
    gender CHAR(1),
    phone_number VARCHAR(15),
    email VARCHAR(100) UNIQUE
);

CREATE TABLE Appointments (
    appointment_id INT PRIMARY KEY,
    doctor_id INT NOT NULL,
    patient_id INT NOT NULL,
    appointment_datetime DATETIME NOT NULL,
    end_datetime DATETIME NOT NULL,
    reason TEXT,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),
    CONSTRAINT CHK_AppointmentTime CHECK (appointment_datetime < end_datetime),
    CONSTRAINT CHK_FutureAppointment CHECK (appointment_datetime >= CURRENT_TIMESTAMP)
);
"""

# SQLite's CURRENT_TIMESTAMP requires a specific format or function call
# For simplicity in this demo, we'll replace it with a fixed timestamp
# In a real application, you'd use a proper date/time function for your specific SQL dialect
create_tables_sql = create_tables_sql.replace('CURRENT_TIMESTAMP', "'" + datetime.now().strftime('%Y-%m-%d %H:%M:%S') + "'")
```

```
cursor.executescript(create_tables_sql)
conn.commit()
print("Tables created successfully.")
```

```
Tables created successfully.
```

Insert Sample Data

```
# Sample Doctors
doctors_data = [
    (1, 'Alice', 'Smith', 'Cardiology', '555-1234', 'alice.s@example.com'),
    (2, 'Bob', 'Johnson', 'Pediatrics', '555-5678', 'bob.j@example.com'),
    (3, 'Charlie', 'Brown', 'Dermatology', '555-9012', 'charlie.b@example.com'),
]
cursor.executemany("INSERT INTO Doctors VALUES (?, ?, ?, ?, ?, ?)", doctors_data)

# Sample Patients
patients_data = [
    (101, 'Eve', 'Davis', '1985-03-15', 'F', '555-1111', 'eve.d@example.com'),
    (102, 'Frank', 'Miller', '1990-07-22', 'M', '555-2222', 'frank.m@example.com'),
    (103, 'Grace', 'Wilson', '1978-11-01', 'F', '555-3333', 'grace.w@example.com'),
    (104, 'Heidi', 'White', '2000-01-01', 'F', '555-4444', 'heidi.w@example.com'),
]
cursor.executemany("INSERT INTO Patients VALUES (?, ?, ?, ?, ?, ?, ?)", patients_data)

# Sample Appointments
```

```
# Sample Appointments
now = datetime.now()
appointments_data = [
    (1001, 1, 101, (now + timedelta(days=5, hours=10)).strftime('%Y-%m-%d %H:%M:%S'), (now + timedelta(days=5, hours=11)).strftime('%Y-%m-%d %H:%M:%S'), 'Rout
    (1002, 1, 102, (now + timedelta(days=6, hours=14)).strftime('%Y-%m-%d %H:%M:%S'), (now + timedelta(days=6, hours=15)).strftime('%Y-%m-%d %H:%M:%S'), 'Follo
    (1003, 2, 101, (now + timedelta(days=7, hours=9)).strftime('%Y-%m-%d %H:%M:%S'), (now + timedelta(days=7, hours=9, minutes=45)).strftime('%Y-%m-%d %H:%M:%
    (1004, 3, 103, (now + timedelta(days=8, hours=11)).strftime('%Y-%m-%d %H:%M:%S'), (now + timedelta(days=8, hours=12)).strftime('%Y-%m-%d %H:%M:%S'), 'Skin
    (1005, 1, 104, (now + timedelta(days=9, hours=16)).strftime('%Y-%m-%d %H:%M:%S'), (now + timedelta(days=9, hours=16, minutes=30)).strftime('%Y-%m-%d %H:%M
    (1006, 2, 104, (now + timedelta(days=10, hours=11)).strftime('%Y-%m-%d %H:%M:%S'), (now + timedelta(days=10, hours=12)).strftime('%Y-%m-%d %H:%M:%S'), 'Ch
]
cursor.executemany("INSERT INTO Appointments VALUES (?, ?, ?, ?, ?, ?)", appointments_data)

conn.commit()
print("Sample data inserted successfully.")
```

```
Sample data inserted successfully.
```

Execute Queries

```
# Query 1: List appointments for a doctor (e.g., doctor_id = 1)
query_1 = """
SELECT
    A.appointment_id,
    A.appointment_datetime,
    A.end_datetime,
    A.reason,
```

```
    Appointments AS A
JOIN
    Patients AS P ON A.patient_id = P.patient_id
WHERE
    A.doctor_id = 1
ORDER BY
    A.appointment_datetime;
"""
cursor.execute(query_1)
results_1 = cursor.fetchall()

print("\n--- Query 1: Appointments for Doctor ID 1 ---")
df_q1 = pd.DataFrame(results_1, columns=[desc[0] for desc in cursor.description])
display(df_q1)


# Query 2: Retrieve patient history by ID (e.g., patient_id = 101)
query_2 = """
SELECT
    A.appointment_id,
    A.appointment_datetime,
    A.end_datetime,
    A.reason,
    D.first_name AS doctor_first_name,
    D.last_name AS doctor_last_name,
    D.specialty
FROM
    Appointments AS A
JOIN
```

```
# Query 3: Count distinct patients per doctor using joins
query_3 = """
SELECT
    D.doctor_id,
    D.first_name AS doctor_first_name,
    D.last_name AS doctor_last_name,
    D.specialty,
    COUNT(DISTINCT A.patient_id) AS distinct_patients_count
FROM
    Doctors AS D
LEFT JOIN
    Appointments AS A ON D.doctor_id = A.doctor_id
GROUP BY
    D.doctor_id, D.first_name, D.last_name, D.specialty
ORDER BY
    distinct_patients_count DESC;
"""
cursor.execute(query_3)
results_3 = cursor.fetchall()

print("\n--- Query 3: Distinct Patients per Doctor ---")
df_q3 = pd.DataFrame(results_3, columns=[desc[0] for desc in cursor.description])
display(df_q3)


# Close the connection
conn.close()
print("\nDatabase connection closed.")
```

Output :

```
--- Query 1: Appointments for Doctor ID 1 ---
```

	appointment_id	appointment_datetime	end_datetime	reason	patient_first_name	patient_last_name
0	1001	2026-04-06 13:53:49	2026-04-06 14:53:49	Routine Checkup	Eve	Davis
1	1002	2026-04-07 17:53:49	2026-04-07 18:53:49	Follow-up	Frank	Miller
2	1005	2026-04-10 19:53:49	2026-04-10 20:23:49	Annual Physical	Heidi	White

```
--- Query 2: Patient History for Patient ID 101 ---
```

	appointment_id	appointment_datetime	end_datetime	reason	doctor_first_name	doctor_last_name	specialty
0	1003	2026-04-08 12:53:49	2026-04-08 13:38:49	Pediatric Consultation	Bob	Johnson	Pediatrics
1	1001	2026-04-06 13:53:49	2026-04-06 14:53:49	Routine Checkup	Alice	Smith	Cardiology

```
--- Query 3: Distinct Patients per Doctor ---
```

	doctor_id	doctor_first_name	doctor_last_name	specialty	distinct_patients_count
0	1	Alice	Smith	Cardiology	3
1	2	Bob	Johnson	Pediatrics	2
2	3	Charlie	Brown	Dermatology	1

```
Database connection closed.
```

Explanation : **SQL Schema Creation:** We started by defining a normalized Hospital Management System schema including Doctors, Patients, and Appointments tables, complete with primary keys, foreign keys, and CHECK constraints for data integrity.
SQL Queries: I provided three SQL queries to demonstrate functionality:Listing appointments for a specific doctor.Retrieving a patient's historical appointments.
Counting distinct patients seen by each doctor.

Task 2 – Real-Time Application: Online Shopping Database
Scenario:
Design a database for an **E-commerce platform**.
Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
  - Retrieve all orders by a user.
  - Find the most purchased product.
  - Calculate total revenue in a given month.
- AI should also suggest **normalization improvements** and **query optimization**.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

Prompt :

> Create a normalized E-commerce database schema with Users, Products, Orders, and OrderDetails tables, then write SQL queries to list all orders by a user, find the most purchased product, and calculate monthly revenue, including suggestions for normalization and query optimization.

Code :

```python
import sqlite3
import pandas as pd
from datetime import datetime, timedelta

# Connect to an in-memory SQLite database
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

print("Connected to in-memory SQLite database.")
```

```
Connected to in-memory SQLite database.
```

Create Tables

```python
create_tables_sql = """
CREATE TABLE Doctors (
    doctor_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    specialty VARCHAR(100),
    phone_number VARCHAR(15),
    email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Patients (
    patient_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE,
    gender CHAR(1),
    phone_number VARCHAR(15),
    email VARCHAR(100) UNIQUE
);

CREATE TABLE Appointments (
    appointment_id INT PRIMARY KEY,
    doctor_id INT NOT NULL,
    patient_id INT NOT NULL,
    appointment_datetime DATETIME NOT NULL,
    end_datetime DATETIME NOT NULL,
    reason TEXT,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),
    CONSTRAINT CHK_AppointmentTime CHECK (appointment_datetime < end_datetime),
    CONSTRAINT CHK_FutureAppointment CHECK (appointment_datetime >= CURRENT_TIMESTAMP)
);
"""

# SQLite's CURRENT_TIMESTAMP requires a specific format or function call
# For simplicity in this demo, we'll replace it with a fixed timestamp
# In a real application, you'd use a proper date/time function for your specific SQL dialect
create_tables_sql = create_tables_sql.replace('CURRENT_TIMESTAMP', "'" + datetime.now().strftime('%Y-%m-%d %H:%M:%S') + "'")

cursor.executescript(create_tables_sql)
```

```
conn.commit()
print("Tables created successfully.")
```

```
Tables created successfully.
```

Insert Sample Data

```
# Sample Doctors
doctors_data = [
    (1, 'Alice', 'Smith', 'Cardiology', '555-1234', 'alice.s@example.com'),
    (2, 'Bob', 'Johnson', 'Pediatrics', '555-5678', 'bob.j@example.com'),
    (3, 'Charlie', 'Brown', 'Dermatology', '555-9012', 'charlie.b@example.com'),
]
cursor.executemany("INSERT INTO Doctors VALUES (?, ?, ?, ?, ?, ?)", doctors_data)

# Sample Patients
patients_data = [
    (101, 'Eve', 'Davis', '1985-03-15', 'F', '555-1111', 'eve.d@example.com'),
    (102, 'Frank', 'Miller', '1990-07-22', 'M', '555-2222', 'frank.m@example.com'),
    (103, 'Grace', 'Wilson', '1978-11-01', 'F', '555-3333', 'grace.w@example.com'),
    (104, 'Heidi', 'White', '2000-01-01', 'F', '555-4444', 'heidi.w@example.com'),
]
cursor.executemany("INSERT INTO Patients VALUES (?, ?, ?, ?, ?, ?, ?)", patients_data)

# Sample Appointments
now = datetime.now()
```

```
# Sample Appointments
now = datetime.now()
appointments_data = [
    (1001, 1, 101, (now + timedelta(days=5, hours=10)).strftime('%Y-%m-%d %H:%M:%S'), (now + timedelta(days=5, hours=11)).strftime('%Y-%m-%d %H:%M:%S'), 'Rout
    (1002, 1, 102, (now + timedelta(days=6, hours=14)).strftime('%Y-%m-%d %H:%M:%S'), (now + timedelta(days=6, hours=15)).strftime('%Y-%m-%d %H:%M:%S'), 'Foll
    (1003, 2, 101, (now + timedelta(days=7, hours=9)).strftime('%Y-%m-%d %H:%M:%S'), (now + timedelta(days=7, hours=9, minutes=45)).strftime('%Y-%m-%d %H:%M:%
    (1004, 3, 103, (now + timedelta(days=8, hours=11)).strftime('%Y-%m-%d %H:%M:%S'), (now + timedelta(days=8, hours=12)).strftime('%Y-%m-%d %H:%M:%S'), 'Skin
    (1005, 1, 104, (now + timedelta(days=9, hours=16)).strftime('%Y-%m-%d %H:%M:%S'), (now + timedelta(days=9, hours=16, minutes=30)).strftime('%Y-%m-%d %H:%M
    (1006, 2, 104, (now + timedelta(days=10, hours=11)).strftime('%Y-%m-%d %H:%M:%S'), (now + timedelta(days=10, hours=12)).strftime('%Y-%m-%d %H:%M:%S'), 'Ch
]
cursor.executemany("INSERT INTO Appointments VALUES (?, ?, ?, ?, ?, ?)", appointments_data)

conn.commit()
print("Sample data inserted successfully.")
```

```
Sample data inserted successfully.
```

+ Code + Text

Execute Queries

```
# Query 1: List appointments for a doctor (e.g., doctor_id = 1)
query_1 = """
SELECT
    A.appointment_id,
    A.appointment_datetime,
    A.end_datetime,
    A.reason,
```

```python
"""
cursor.execute(query_1)
results_1 = cursor.fetchall()

print("\n--- Query 1: Appointments for Doctor ID 1 ---")
df_q1 = pd.DataFrame(results_1, columns=[desc[0] for desc in cursor.description])
display(df_q1)


# Query 2: Retrieve patient history by ID (e.g., patient_id = 101)
query_2 = """
SELECT
    A.appointment_id,
    A.appointment_datetime,
    A.end_datetime,
    A.reason,
    D.first_name AS doctor_first_name,
    D.last_name AS doctor_last_name,
    D.specialty
FROM
    Appointments AS A
JOIN
    Doctors AS D ON A.doctor_id = D.doctor_id
WHERE
    A.patient_id = 101
ORDER BY
    A.appointment_datetime DESC;
"""
```

```python
conn.close()
print("\nDatabase connection closed.")
```

```
--- Query 1: Appointments for Doctor ID 1 ---
```

	appointment_id	appointment_datetime	end_datetime	reason	patient_first_name	patient_last_name
0	1001	2026-04-06 13:53:49	2026-04-06 14:53:49	Routine Checkup	Eve	Davis
1	1002	2026-04-07 17:53:49	2026-04-07 18:53:49	Follow-up	Frank	Miller
2	1005	2026-04-10 19:53:49	2026-04-10 20:23:49	Annual Physical	Heidi	White

```
--- Query 2: Patient History for Patient ID 101 ---
```

	appointment_id	appointment_datetime	end_datetime	reason	doctor_first_name	doctor_last_name	specialty
0	1003	2026-04-08 12:53:49	2026-04-08 13:38:49	Pediatric Consultation	Bob	Johnson	Pediatrics
1	1001	2026-04-06 13:53:49	2026-04-06 14:53:49	Routine Checkup	Alice	Smith	Cardiology

```
--- Query 3: Distinct Patients per Doctor ---
```

	doctor_id	doctor_first_name	doctor_last_name	specialty	distinct_patients_count
0	1	Alice	Smith	Cardiology	3
1	2	Bob	Johnson	Pediatrics	2
2	3	Charlie	Brown	Dermatology	1

Explanation : The E-commerce database schema, including the Users, Products, Orders, and OrderDetails tables, as well as the requested SQL queries for listing orders, finding the most purchased product, and calculating monthly revenue, have now been successfully added to the notebook. I've also included detailed explanations on normalization and query optimization techniques.

Task 3 – Library Database

Task:

Design schema for a **Library Management System**.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:

- Retrieve all books currently issued.
- Find overdue books (loan date > 30 days).
- Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

Prompt :

Create a normalized Library Management System schema with Books, Members, and Loans tables, then write SQL queries to list issued books, find overdue loans (>30 days), and count books per member, including indexing suggestions for faster queries.

Code :

```
import sqlite3
import pandas as pd
from datetime import datetime, timedelta

# Connect to an in-memory SQLite database
conn_lib = sqlite3.connect(':memory:')
cursor_lib = conn_lib.cursor()

create_library_tables_sql = """
-- Create the Books table
CREATE TABLE Books (
    book_id INT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(100) NOT NULL,
    ISBN VARCHAR(20) UNIQUE NOT NULL,
    publication_year INT CHECK (publication_year > 1000),
    genre VARCHAR(50)
);

-- Create the Members table
CREATE TABLE Members (
    member_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone_number VARCHAR(15) UNIQUE
);
```

```
-- Create the Loans table
CREATE TABLE Loans (
    loan_id INT PRIMARY KEY,
    book_id INT NOT NULL,
    member_id INT NOT NULL,
    loan_date DATE NOT NULL,
    due_date DATE NOT NULL,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id),
    CONSTRAINT CHK_LoanDates CHECK (loan_date <= due_date)
);
"""

cursor_lib.executescript(create_library_tables_sql)
conn_lib.commit()
print("Library tables created successfully.")

# Insert Sample Data

# Sample Books
books_data = [
    (1, 'The Great Gatsby', 'F. Scott Fitzgerald', '978-0743273565', 1925, 'Classic'),
    (2, '1984', 'George Orwell', '978-0451524935', 1949, 'Dystopian'),
    (3, 'To Kill a Mockingbird', 'Harper Lee', '978-0061120084', 1960, 'Classic'),
    (4, 'Pride and Prejudice', 'Jane Austen', '978-0141439518', 1813, 'Romance'),
    (5, 'The Hobbit', 'J.R.R. Tolkien', '978-0345339683', 1937, 'Fantasy')
```

```
]
cursor_lib.executemany("INSERT INTO Books VALUES (?, ?, ?, ?, ?, ?)", books_data)

# Sample Members
members_data = [
    (101, 'Alice', 'Smith', 'alice.s@example.com', '555-1001'),
    (102, 'Bob', 'Johnson', 'bob.j@example.com', '555-1002'),
    (103, 'Charlie', 'Brown', 'charlie.b@example.com', '555-1003')
]
cursor_lib.executemany("INSERT INTO Members VALUES (?, ?, ?, ?, ?)", members_data)

# Sample Loans
now = datetime.now()
loans_data = [
    (10001, 1, 101, (now - timedelta(days=10)).strftime('%Y-%m-%d'), (now + timedelta(days=20)).strftime('%Y-%m-%d'), None), # Alice borrowed The Great Gatsby
    (10002, 2, 102, (now - timedelta(days=50)).strftime('%Y-%m-%d'), (now - timedelta(days=20)).strftime('%Y-%m-%d'), None), # Bob borrowed 1984, overdue
    (10003, 3, 101, (now - timedelta(days=5)).strftime('%Y-%m-%d'), (now + timedelta(days=25)).strftime('%Y-%m-%d'), None), # Alice borrowed To Kill a Mocking
    (10004, 4, 103, (now - timedelta(days=15)).strftime('%Y-%m-%d'), (now - timedelta(days=5)).strftime('%Y-%m-%d'), (now - timedelta(days=1)).strftime('%Y-%m
    (10005, 5, 102, (now - timedelta(days=20)).strftime('%Y-%m-%d'), (now + timedelta(days=10)).strftime('%Y-%m-%d'), None) # Bob borrowed The Hobbit, not retu
]
cursor_lib.executemany("INSERT INTO Loans VALUES (?, ?, ?, ?, ?, ?)", loans_data)

conn_lib.commit()
print("Sample library data inserted successfully.")
```

```
Library tables created successfully.
Sample library data inserted successfully.
```

```python
# Query 1: List all currently issued books
query_issued_books = """
SELECT
    B.title,
    B.author,
    B.ISBN,
    M.first_name AS member_first_name,
    M.last_name AS member_last_name,
    L.loan_date,
    L.due_date
FROM
    Loans AS L
JOIN
    Books AS B ON L.book_id = B.book_id
JOIN
    Members AS M ON L.member_id = M.member_id
WHERE
    L.return_date IS NULL
ORDER BY
    L.loan_date DESC;
"""
cursor_lib.execute(query_issued_books)
results_issued_books = cursor_lib.fetchall()

print("\n--- Query 1: Currently Issued Books ---")
df_issued_books = pd.DataFrame(results_issued_books, columns=[desc[0] for desc in cursor_lib.description])
display(df_issued_books)
```

```python
results_issued_books = cursor_lib.fetchall()

print("\n--- Query 1: Currently Issued Books ---")
df_issued_books = pd.DataFrame(results_issued_books, columns=[desc[0] for desc in cursor_lib.description])
display(df_issued_books)


# Query 2: Find overdue loans (>30 days past due_date)
query_overdue_loans = """
SELECT
    B.title,
    B.author,
    M.first_name AS member_first_name,
    M.last_name AS member_last_name,
    L.loan_date,
    L.due_date,
    JULIANDAY('now') - JULIANDAY(L.due_date) AS days_overdue
FROM
    Loans AS L
JOIN
    Books AS B ON L.book_id = B.book_id
JOIN
    Members AS M ON L.member_id = M.member_id
WHERE
    L.return_date IS NULL AND L.due_date < DATE('now') AND JULIANDAY('now') - JULIANDAY(L.due_date) > 30
ORDER BY
    days_overdue DESC;
"""
```

```python
# Query 3: Count books per member (currently on loan)
query_books_per_member = """
SELECT
    M.first_name,
    M.last_name,
    COUNT(L.book_id) AS books_on_loan
FROM
    Members AS M
LEFT JOIN
    Loans AS L ON M.member_id = L.member_id
WHERE
    L.return_date IS NULL OR L.loan_id IS NULL -- Include members with no loans
GROUP BY
    M.member_id, M.first_name, M.last_name
ORDER BY
    books_on_loan DESC;
"""
cursor_lib.execute(query_books_per_member)
results_books_per_member = cursor_lib.fetchall()

print("\n--- Query 3: Books on Loan per Member ---")
df_books_per_member = pd.DataFrame(results_books_per_member, columns=[desc[0] for desc in cursor_lib.description])
display(df_books_per_member)

# Close the connection
conn_lib.close()
print("\nLibrary database connection closed.")
```

```
--- Query 1: Currently Issued Books ---
```

	title	author	ISBN	member_first_name	member_last_name	loan_date	due_date
0	To Kill a Mockingbird	Harper Lee	978-0061120084	Alice	Smith	2026-03-27	2026-04-26
1	The Great Gatsby	F. Scott Fitzgerald	978-0743273565	Alice	Smith	2026-03-22	2026-04-21
2	The Hobbit	J.R.R. Tolkien	978-0345339683	Bob	Johnson	2026-03-12	2026-04-11
3	1984	George Orwell	978-0451524935	Bob	Johnson	2026-02-10	2026-03-12

```
--- Query 2: Overdue Loans (more than 30 days) ---
```

title	author	member_first_name	member_last_name	loan_date	due_date	days_overdue

```
--- Query 3: Books on Loan per Member ---
```

	first_name	last_name	books_on_loan
0	Alice	Smith	2
1	Bob	Johnson	2

```
Library database connection closed.
```

Explanation : This query successfully listed all books that are currently on loan. The results show the book title, author, ISBN, the member who borrowed it, and the loan and due dates

Task 4: Optimize Queries

Instructions:

- Use AI tools to analyze query efficiency.
- Optimize inefficient queries using joins, indexes, or reducing subqueries.
- Test optimized queries for correctness.

Starter Code Example:

-- Original query

SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');

-- Optimized query

SELECT b.*

FROM Books b

JOIN Authors a ON b.author_id = a.author_id

WHERE a.country = 'UK';

Expected Output:

- Both queries return the same result: all books written by UK authors.

- Optimized query performs faster for larger datasets.

Prompt :

Analyze SQL queries in a Library or E-commerce database, replace inefficient subqueries with joins, suggest indexes, and verify correctness for faster performance.

Code:

```
import pandas as pd

# Query 1: Find users who have purchased a specific product (using subquery)
query_1_subquery = """
SELECT
    U.user_id, U.username, U.email
FROM
    Users AS U
WHERE
    U.user_id IN (
        SELECT O.user_id
        FROM Orders AS O
        JOIN OrderDetails AS OD ON O.order_id = OD.order_id
        JOIN Products AS P ON OD.product_id = P.product_id
        WHERE P.product_name = 'Laptop Pro'
    );
"""

cursor_ecommerce.execute(query_1_subquery)
results_1_subquery = cursor_ecommerce.fetchall()
print("\n--- Query 1 (Subquery): Users who purchased 'Laptop Pro' ---")
df_q1_subquery = pd.DataFrame(results_1_subquery, columns=[desc[0] for desc in cursor_ecommerce.description])
display(df_q1_subquery)

# Query 1 Optimized: Find users who have purchased a specific product (using JOIN)
query_1_join = """
SELECT DISTINCT
```

```
JOIN
    Orders AS O ON U.user_id = O.user_id
JOIN
    OrderDetails AS OD ON O.order_id = OD.order_id
JOIN
    Products AS P ON OD.product_id = P.product_id
WHERE
    P.product_name = 'Laptop Pro';
"""

cursor_ecommerce.execute(query_1_join)
results_1_join = cursor_ecommerce.fetchall()
print("\n--- Query 1 (JOIN Optimized): Users who purchased 'Laptop Pro' ---")
df_q1_join = pd.DataFrame(results_1_join, columns=[desc[0] for desc in cursor_ecommerce.description])
display(df_q1_join)


# Query 2: List products that have never been ordered (using subquery NOT IN)
query_2_subquery = """
SELECT
    P.product_id, P.product_name, P.category
FROM
    Products AS P
WHERE
    P.product_id NOT IN (
        SELECT DISTINCT product_id
        FROM OrderDetails
```

```python
cursor_ecommerce.execute(query_2_subquery)
results_2_subquery = cursor_ecommerce.fetchall()
print("\n--- Query 2 (Subquery): Products never ordered ---")
df_q2_subquery = pd.DataFrame(results_2_subquery, columns=[desc[0] for desc in cursor_ecommerce.description])
display(df_q2_subquery)

# Query 2 Optimized: List products that have never been ordered (using LEFT JOIN / IS NULL)
query_2_join = """
SELECT
    P.product_id, P.product_name, P.category
FROM
    Products AS P
LEFT JOIN
    OrderDetails AS OD ON P.product_id = OD.product_id
WHERE
    OD.product_id IS NULL;
"""

cursor_ecommerce.execute(query_2_join)
results_2_join = cursor_ecommerce.fetchall()
print("\n--- Query 2 (JOIN Optimized): Products never ordered ---")
df_q2_join = pd.DataFrame(results_2_join, columns=[desc[0] for desc in cursor_ecommerce.description])
display(df_q2_join)


# Query 3: Get the total amount spent by each user
query_3 = """
```

```python
cursor_ecommerce.execute(query_4_subquery_correlated)
results_4_subquery_correlated = cursor_ecommerce.fetchall()
print("\n--- Query 4 (Correlated Subquery): Latest order for each user ---")
df_q4_subquery_correlated = pd.DataFrame(results_4_subquery_correlated, columns=[desc[0] for desc in cursor_ecommerce.descripti
display(df_q4_subquery_correlated)

# Query 4 Optimized: Find the latest order for each user (using Common Table Expression (CTE) and ROW_NUMBER)
# Note: SQLite doesn't directly support ROW_NUMBER, so we'll use a standard approach.
# For SQLite, a common pattern involves a subquery to find max date per user and then join back.
query_4_optimized_sqlite = """
SELECT
    O.order_id, O.user_id, O.order_date, O.total_amount
FROM
    Orders AS O
JOIN (
    SELECT
        user_id,
        MAX(order_date) AS max_order_date
    FROM
        Orders
    GROUP BY
        user_id
) AS latest_orders ON O.user_id = latest_orders.user_id AND O.order_date = latest_orders.max_order_date;
"""
cursor_ecommerce.execute(query_4_optimized_sqlite)
results_4_optimized_sqlite = cursor_ecommerce.fetchall()
print("\n--- Query 4 (Optimized SQLite): Latest order for each user ---")
```

Output :

```
--- Query 1 (Subquery): Users who purchased 'Laptop Pro' ---
   user_id  username            email
0        1  john_doe  john.doe@example.com
1        2  jane_smith  jane.smith@example.com

--- Query 1 (JOIN Optimized): Users who purchased 'Laptop Pro' ---
   user_id  username            email
0        1  john_doe  john.doe@example.com
1        2  jane_smith  jane.smith@example.com

--- Query 2 (Subquery): Products never ordered ---
  product_id  product_name  category

--- Query 2 (JOIN Optimized): Products never ordered ---
  product_id  product_name  category

--- Query 3: Total amount spent by each user ---
    username  total_spent
0  jane_smith        1450
```

Explanation : The E-commerce tables have been successfully created and populated with sample data. Now, let's move on to the exciting part: analyzing and optimizing some SQL queries for this database. I'll provide examples of common queries, discuss potential inefficiencies, and then demonstrate how to refactor them using joins and suggest appropriate indexes to boost performance. This will cover identifying issues, proposing solutions, and ensuring correctness.

Task 5: University Course Registration

Scenario:

SR University needs a **course registration system** for students enrolling in different courses under faculty supervision.

- Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.
- Define relationships:
  - Each student can register for multiple courses.
  - Each course is taught by a faculty member.
- Write SQL queries to:
  - List all students enrolled in a specific course.
  - Find faculty members teaching more than 2 courses.
  - Retrieve courses with the highest number of registrations.

Prompt :

Create a normalized University Course Registration schema with Students, Courses, Faculty, and Registrations tables, then write SQL queries to list students in a course, find faculty teaching more than two courses, and get courses with the highest registrations.

Code :

```
# Query 1: List students in a specific course (e.g., 'CS101')
query_students_in_course = """
SELECT
    S.first_name,
    S.last_name,
    S.email,
    C.course_code,
    C.course_title
FROM
    Students AS S
JOIN
    Registrations AS R ON S.student_id = R.student_id
JOIN
    Courses AS C ON R.course_id = C.course_id
WHERE
    C.course_code = 'CS101'
ORDER BY
    S.last_name, S.first_name;
"""
cursor_uni.execute(query_students_in_course)
results_students_in_course = cursor_uni.fetchall()

print("\n--- Query 1: Students in 'CS101' ---")
df_students_in_course = pd.DataFrame(results_students_in_course, columns=[desc[0] for desc in cursor_uni.description])
display(df_students_in_course)
```

```
# Query 2: Find faculty teaching more than two courses
query_faculty_teaching_multiple_courses = """
SELECT
    F.first_name,
    F.last_name,
    F.department,
    COUNT(C.course_id) AS num_courses_taught
FROM
    Faculty AS F
JOIN
    Courses AS C ON F.faculty_id = C.faculty_id
GROUP BY
    F.faculty_id, F.first_name, F.last_name, F.department
HAVING
    COUNT(C.course_id) > 2
ORDER BY
    num_courses_taught DESC;
"""
cursor_uni.execute(query_faculty_teaching_multiple_courses)
results_faculty_teaching_multiple_courses = cursor_uni.fetchall()

print("\n--- Query 2: Faculty Teaching More Than Two Courses ---")
df_faculty_teaching_multiple_courses = pd.DataFrame(results_faculty_teaching_multiple_courses, columns=[desc[0] for desc in cursor_uni.description])
display(df_faculty_teaching_multiple_courses)


# Query 3: Get courses with the highest registrations
query_courses_highest_registrations = """
```

```
SELECT
    C.course_code,
    C.course_title,
    F.first_name AS faculty_first_name,
    F.last_name AS faculty_last_name,
    COUNT(R.student_id) AS total_registrations
FROM
    Courses AS C
LEFT JOIN
    Registrations AS R ON C.course_id = R.course_id
LEFT JOIN
    Faculty AS F ON C.faculty_id = F.faculty_id
GROUP BY
    C.course_id, C.course_code, C.course_title, F.first_name, F.last_name
ORDER BY
    total_registrations DESC
LIMIT 3; -- Get the top 3 courses
"""
cursor_uni.execute(query_courses_highest_registrations)
results_courses_highest_registrations = cursor_uni.fetchall()

print("\n--- Query 3: Courses with the Highest Registrations ---")
df_courses_highest_registrations = pd.DataFrame(results_courses_highest_registrations, columns=[desc[0] for desc in cursor_uni.description])
display(df_courses_highest_registrations)

# Close the connection
conn_uni.close()
```

```
list: results_faculty_teaching_multiple_courses
(1 item) [('Dr. Emily', 'White', 'Computer Science', 3)]
```

Output :

```
--- Query 1: Students in 'CS101' ---
```

	first_name	last_name	email	course_code	course_title
0	Bob	Johnson	bob.j@university.edu	CS101	Introduction to Programming
1	Diana	Prince	diana.p@university.edu	CS101	Introduction to Programming
2	Alice	Smith	alice.s@university.edu	CS101	Introduction to Programming

```
--- Query 2: Faculty Teaching More Than Two Courses ---
```

	first_name	last_name	department	num_courses_taught
0	Dr. Emily	White	Computer Science	3

```
--- Query 3: Courses with the Highest Registrations ---
```

	course_code	course_title	faculty_first_name	faculty_last_name	total_registrations
0	CS101	Introduction to Programming	Dr. Emily	White	3
1	MA201	Calculus I	Prof. Mark	Green	1
2	PH101	General Physics	Dr. Sarah	Black	1

```
University database connection closed.
```

Explanation :University Course Registration schema including Students, Courses, Faculty, and Registrations tables. Then, I will provide SQL queries to list students in a course, find faculty teaching more than two courses, and identify courses with the highest registrations. I'll also include indexing suggestions for better performance.